Caltech

Machine Learning & Data Mining

CMS/CS/CNS/EE 155

Lecture 1:

Administrivia & Basics

Course Info

- Lecture (Tu/Th)
  - 2:30pm - 4pm in Beckman Institute Auditorium

- Recitation
  - Evening,
  - As needed
  - Usually 45-60 minutes
  - **First one on Thursday! (Introduction to Python)**

Staff

Ryan Lin (Head TA)

Ryan Hu

Idil Turasi

Amanda Wang

Isha Goswami

Olivia Wang

Harry Chen

Kailen Hargenrader

Divin Irakiza

Madeline Egan

Sophia Stiles

Tanvi Ganapathy

Allison Xin

Anna Szczuka

Course Breakdown

- 6 Homeworks, ~60% of final grade
  - Due on Tuesdays nights via Gradescope
  - **Homework 1 will be released tonight.**
    - Due Tuesdays @midnight

Plan accordingly w/ other courses!

- 3 Mini-projects, ~30% of final grade

- Final, ~10% of final grade

Regarding Homework 1

- If you have prior experience with CS 156
  - Should be pretty straightforward (4-5 hours)

- If you do not…
  - Might take a while (8-12 hours?)
  - But will catch you up if you make it through
  - Should consider dropping class if too hard

Late Submission Policy

- Up to 48 free late hours
- Specify # late hours used when submitting

Course Etiquette

- Please ask questions during lecture!
  - I might defer some in interest of time
- If you arrive late, or need to leave early, please do so quietly.
- Same collaboration policy as previous 2-3 years

Policy on Large Language Models

Homework assignments

- Students are expected to complete homework assignments based on their understanding of the course material. Student can use LLMs as a resource (e.g., helping with debugging, or grammar checking), but the assignments (including reports and code) should be principally authored by the student.

Projects

- Students are allowed and encouraged to explore tools like LLMs as a way to supplement learning, not as a shortcut. If students use LLMs in their projects, they must explain in the report how LLMs contribute to the project, providing details like the text and code generated by LLMs.

Final exam

- No use of large language models is allowed.

Course Website

- https://sites.google.com/view/cs-155-machine-learning/

- Linked to from my website
  - http://www.yisongyue.com
- Linked to from Canvas

- Up-to-date office hours
- Lecture notes, homework, etc.

Gradescope & Piazza

- Gradescope:
  - https://www.gradescope.com/courses/932378/
  - Submission, Solutions, Grades
- Piazza
  - https://piazza.com/class/m4xf8vvl4206zc/
  - Course announcements, Homework Posted
  - Q&A Forum (use it!)
- Lecture Videos
  - linked from course website

Machine Learning & Data Mining

Computer Algorithm

Process of Converting

Data & Experience

Into Knowledge

Computer Model

Machine Learning vs Data Mining

- **ML focuses more on algorithms**
  - Typically more rigorous
  - Also on analysis (learning theory)
- **DM focuses more on knowledge extraction**
  - Typically uses ML algorithms
  - Knowledge should be human-understandable
- **Huge overlap**

Course Outline

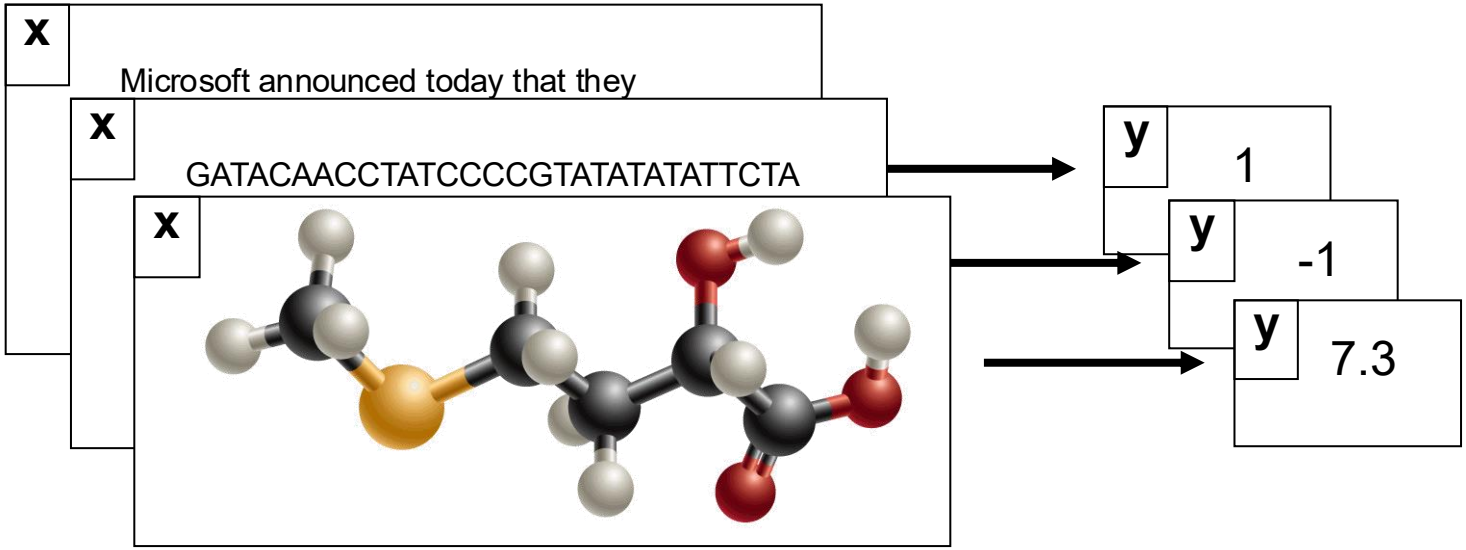
- Supervised Learning
  - 5 weeks
- Unsupervised Learning
  - 2 weeks
- Probabilistic Models
  - 2 weeks

Supervised Learning

- Find function from input space *X* to output space *Y*

$$f : X \square \ Y$$ (sometimes use *h*)

such that the prediction error is low.



Supervised Learning

Data: X

Target Signal: Y

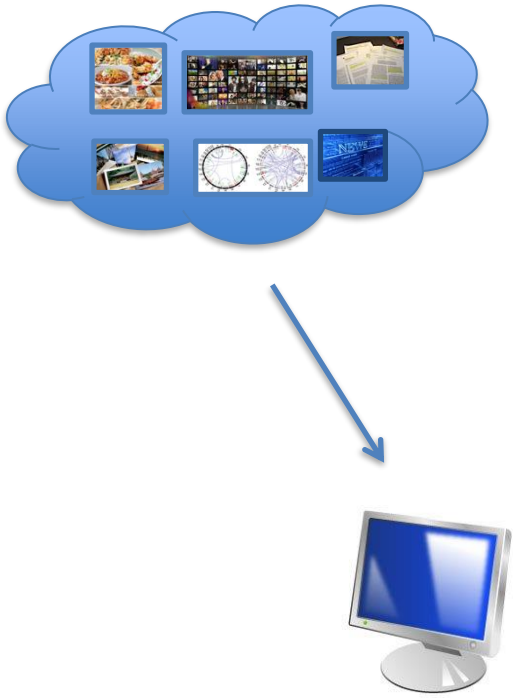
$f(x) \approx y$

Logistic Regression
Neural Networks
Decision Trees
Etc…

(function class or hypothesis class)

Aside: Unsupervised Learning

Data: X



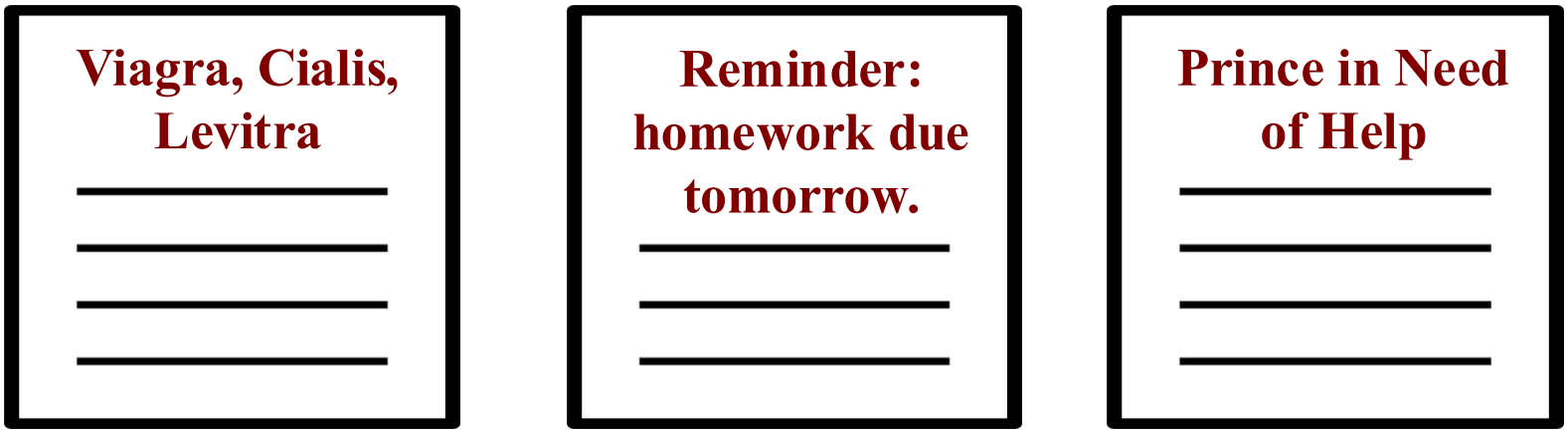
No supervised target!

Learning goal is usually to find low-dimensional “summary” or reconstruction.

More on this later in course. (Precursor to LLMs)

Example: Spam Filtering

- **Goal:** write a program to filter spam.



SPAM!

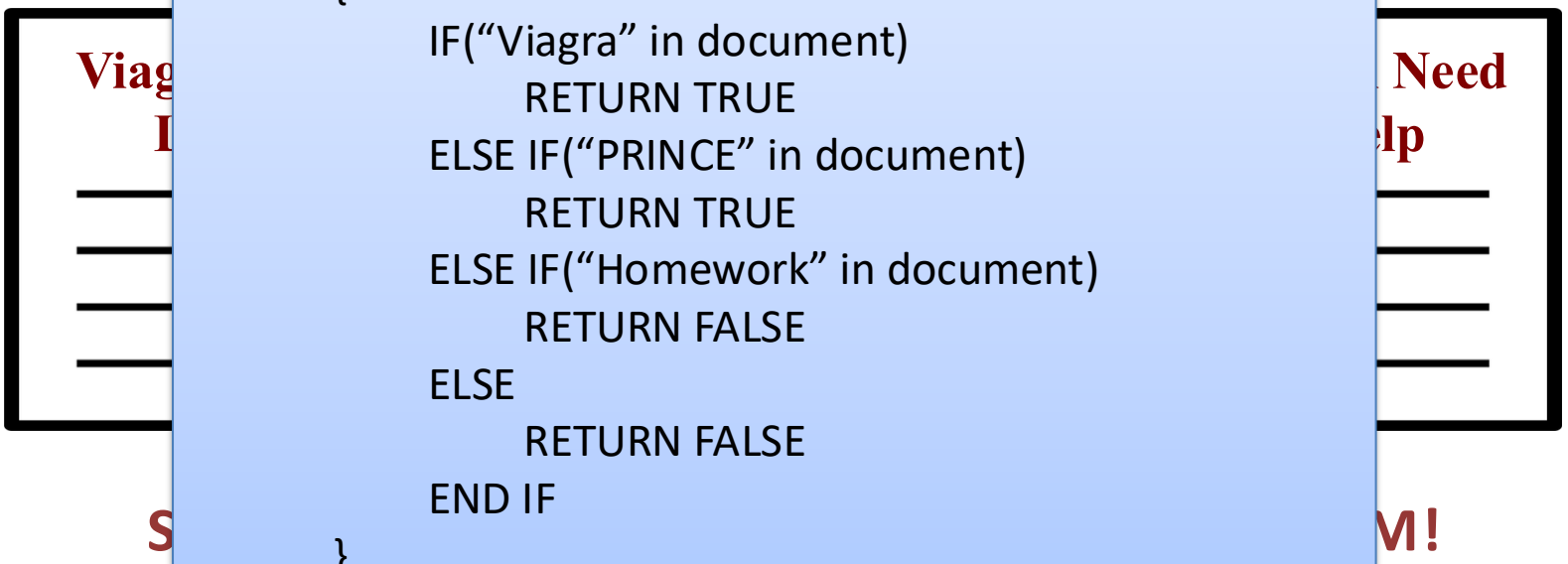
NOT SPAM

SPAM!

Example: Spam Filtering

- **Goal**

```
FUNCTION SpamFilter(string document)
{
        IF(“Viagra” in document)
                RETURN TRUE
        ELSE IF(“PRINCE” in document)
                RETURN TRUE
        ELSE IF(“Homework” in document)
                RETURN FALSE
        ELSE
                RETURN FALSE
        END IF
}
```



Why is Spam Filtering Hard?

- Easy for humans to recognize
- Hard for humans to write down algorithm
- Lots of IF statements!

Machine Learning to the Rescue!

Training Set

SPAM!

SPAM!

NOT SPAM

NOT SPAM

SPAM!

SPAM!

Build a Generic Representation

Run a Generic Learning Algorithm

➔ Classification Model

Labeled by Humans (“Supervision”)

Bag of Words Representation

Training Set

SPAM!

SPAM!

NOT SPAM

NOT SPAM

SPAM!

SPAM!

⋮

Bag of Words

(0,0,0,1,1,1)

(1,0,0,1,0,0)

(1,0,1,0,1,0)

(0,1,1,0,1,0)

(1,0,1,1,0,1)

(1,0,0,0,0,1)

⋮

"Feature Vector"

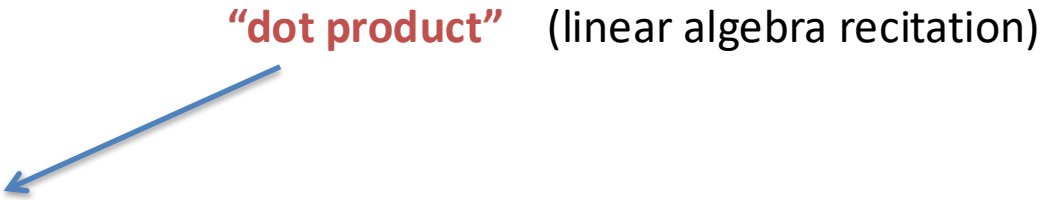
One feature for each word in the vocabulary

In practice 10k-1M

Linear Models

Let x denote the bag-of-words for an email

E.g., x = (1,1,0,0,1,1)



Linear Classifier:

$$f(x|w,b) = \text{sign}(w^T x - b)$$

$$= \text{sign}(w_1 * x_1 + \ldots w_6 * x_6 - b)$$

$$f(x|w,b) = \text{sign}(w^T x - b)$$
$$= \text{sign}(w_1 * x_1 + \ldots w_6 * x_6 - b)$$

w = (1,0,0,1,0,1)
b = 1.5

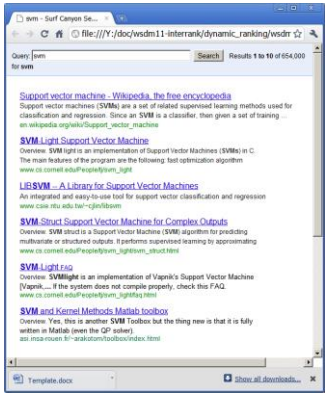
Training Set

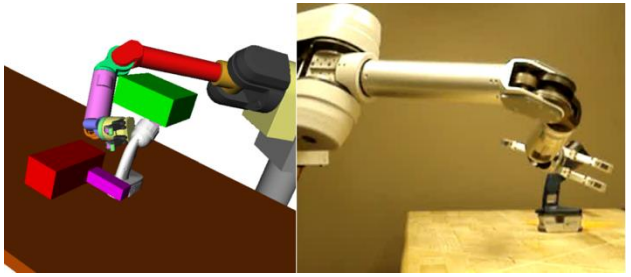
Bag of Words

SPAM!	(0,0,0,1,1,1)	f(x|w,b) = +1
SPAM!	(1,0,0,1,0,0)	f(x|w,b) = +1
NOT SPAM	(1,0,1,0,1,0)	f(x|w,b) = -1
NOT SPAM	(0,1,1,0,1,0)	f(x|w,b) = -1
SPAM!	(1,0,1,1,0,1)	f(x|w,b) = +1
SPAM!	(1,0,0,0,0,1)	f(x|w,b) = +1
⋮	⋮	⋮

Linear Models

- Workhorse of Machine Learning





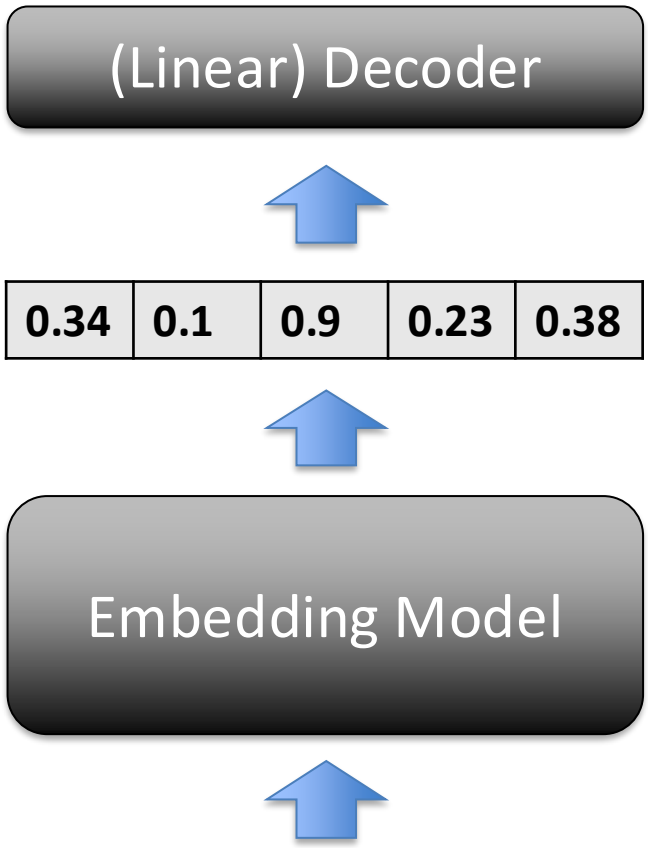
- By end of this lecture, you'll learn 75% how to build basic linear model.

Why Does Machine Learning Work?

- Repeated patterns in the data
  - Typically in the features
  - E.g., “Prince” + “Help” is indicative of spam

- Machine learning will find those patterns
  - Linear model over features
  - E.g., high weight on the words “Prince” & “Help”

Note on Learning Decoders



This lecture so far

Embedding / Representation

(Later in the course)

Raw Input

Two Basic Supervised ML Problems

- **Classification** $f(x \mid w, b) = \text{sign}(w^T x - b)$
  - Predict which class an example belongs to
  - E.g., spam filtering example
- **Regression** $f(x \mid w, b) = w^T x - b$
  - Predict a real value or a probability
  - E.g., probability of being spam
- **Highly inter-related**
  - Train on Regression => Use for Classification

$$f(x|w,b) = w^T x - b$$
$$= w_1 * x_1 + \ldots w_6 * x_6 - b$$

w = (1,0,0,1,0,1)
b = 1.5

Training Set

SPAM!

SPAM!

NOT SPAM

NOT SPAM

SPAM!

SPAM!

⋮

Bag of Words

(0,0,0,1,1,1)

(1,0,0,1,0,0)

(1,0,1,0,1,0)

(0,1,1,0,1,0)

(1,0,1,1,0,1)

(1,0,0,0,0,1)

⋮

$f(x|w,b) = +0.5$

$f(x|w,b) = +0.5$

$f(x|w,b) = -0.5$

$f(x|w,b) = -1.5$

$f(x|w,b) = +1.5$

$f(x|w,b) = +0.5$

⋮

Formal Definitions

- Training set: $S = \{(x_i, y_i)\}_{i=1}^{N}$ $x \in R^D$ $y \in \{-1, +1\}$
- Model class: $f(x \mid w, b) = w^T x - b$ **Linear Models**

aka hypothesis class

- **Goal:** find (w,b) that predicts well on S.
  - How to quantify "well"?

Basic Supervised Learning Recipe

- Training Data: $S = \{(x_i, y_i)\}_{i=1}^{N}$ $x \in R^D$ $y \in \{-1, +1\}$
- Model Class: $f(x \mid w, b) = w^T x - b$ **Linear Models**
- Loss Function: $L(a, b) = (a - b)^2$ **Squared Loss**
- Learning Objective: $\arg\min_{w,b} \sum_{i=1}^{N} L(y_i, f(x_i | w, b))$

Optimization Problem

Loss Function

- Measures penalty of mis-prediction:

- 0/1 Loss: $L(a,b) = 1_{[a \neq b]}$

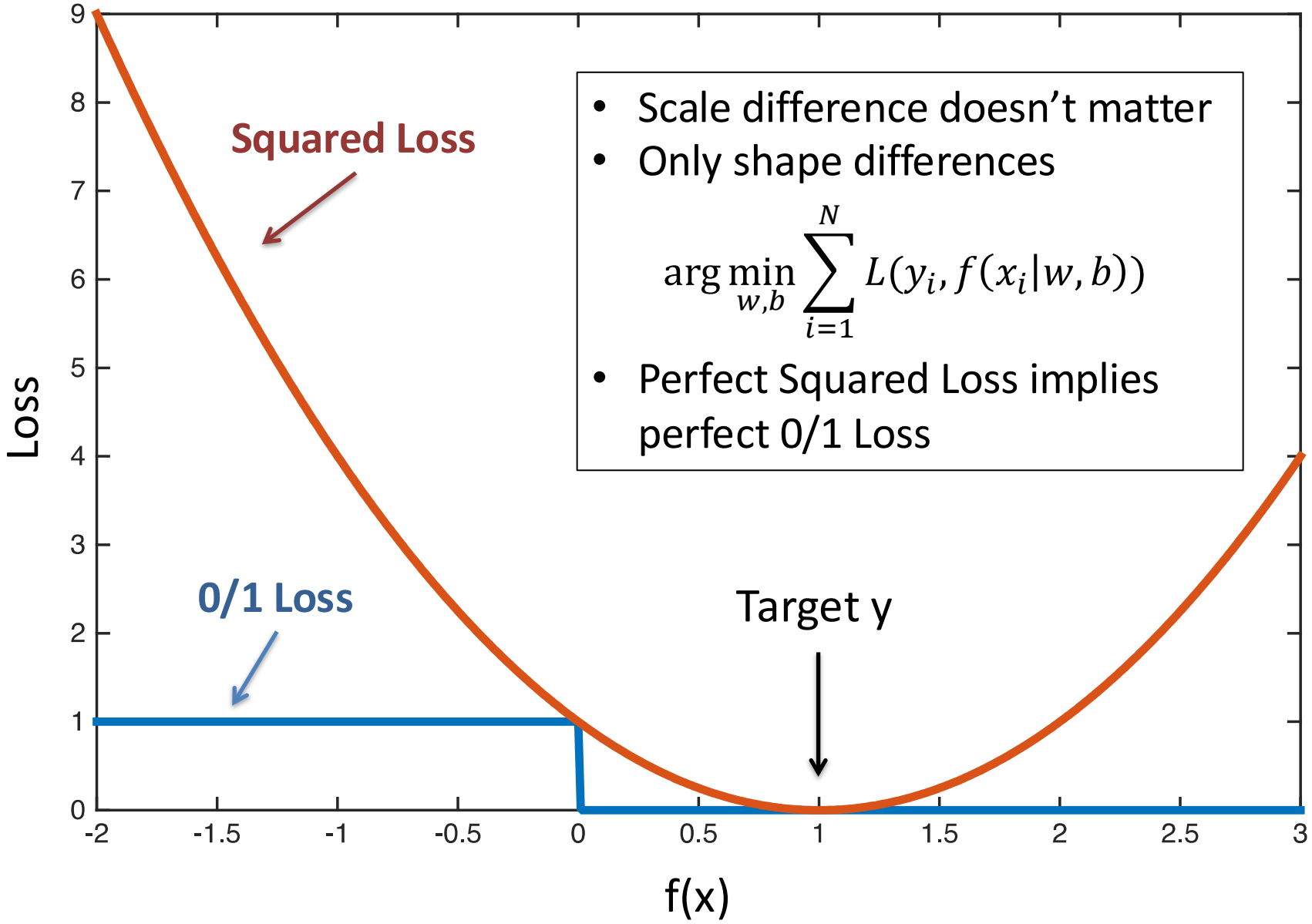
  $L(a,b) = 1_{[\text{sign}(a) \neq \text{sign}(b)]}$

  Classification

- Squared loss: $L(a,b) = (a-b)^2$

  Regression

- Substitute: *a=y*, *b=f(x)*


Squared Loss
0/1 Loss
Target y
Loss
f(x)
9
8
7
6
5
4
3
2
1
0
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
2.5
3
• Scale difference doesn't matter
• Only shape differences
$\arg\min_{w,b} \sum_{i=1}^{N} L(y_i, f(x_i|w,b))$
• Perfect Squared Loss implies perfect 0/1 Loss

$$f(x|w,b) = w^T x - b$$
$$= w_1 * x_1 + \ldots w_6 * x_6 - b$$

w = (0.05, 0.05, -0.68, 0.68, -0.63, 0.68)
b = 0.27

Training Set

	Bag of Words	
SPAM!	(0,0,0,1,1,1)	f(x|w,b) = +1
SPAM!	(1,0,0,1,0,0)	f(x|w,b) = +1
NOT SPAM	(1,0,1,0,1,0)	f(x|w,b) = -1
NOT SPAM	(0,1,1,0,1,0)	f(x|w,b) = -1
SPAM!	(1,0,1,1,0,1)	f(x|w,b) = +1
SPAM!	(1,0,0,0,0,1)	f(x|w,b) = +1

Train using Squared Loss

Learning Algorithm

$$\underset{w,b}{\operatorname{argmin}} \sum_{i=1}^{N} L\left(y_i, f(x_i \mid w, b)\right)$$

- Typically, requires optimization algorithm.

- Simplest: **Gradient Descent**

Loop for T iterations

$$w_{t+1} \leftarrow w_t - \alpha_w \sum_{i=1}^{N} L\left(y_i, f(x_i \mid w_t, b_t)\right)$$

$$b_{t+1} \leftarrow b_t - \alpha_b \sum_{i=1}^{N} L\left(y_i, f(x_i \mid w_t, b_t)\right)$$

Gradient Review

$$\partial_w \sum_{i=1}^{N} L(y_i, f(x_i|w,b))$$

More Details
Next Lecture

$$= \sum_{i=1}^{N} \partial_w L(y_i, f(x_i|w,b))$$

Linearity of Differentiation

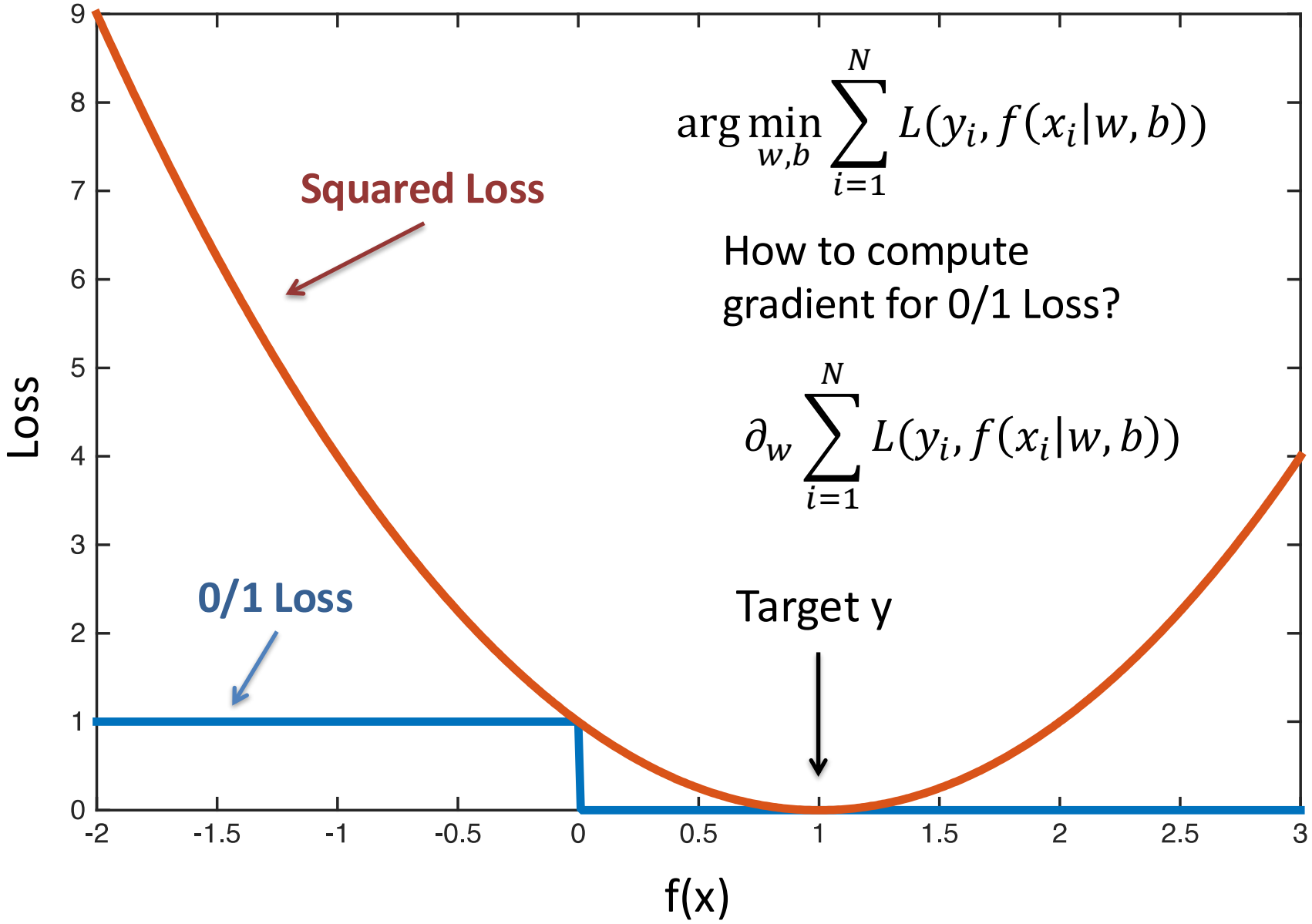
$$= \sum_{i=1}^{N} -2\big(y_i - f(x_i|w,b)\big)\partial_w f(x_i|w,b)$$

$$L(a,b) = (a-b)^2$$

Chain Rule

$$= \sum_{i=1}^{N} -2(y_i - w^T x + b)x$$

$$f(x \mid w,b) = w^T x - b$$



$$\arg\min_{w,b} \sum_{i=1}^{N} L(y_i, f(x_i|w,b))$$

How to compute gradient for 0/1 Loss?

$$\partial_w \sum_{i=1}^{N} L(y_i, f(x_i|w,b))$$

0/1 Loss is Intractable

- 0/1 Loss is flat or discontinuous everywhere
- VERY difficult to optimize
- **Solution:** Optimize smooth surrogate Loss
  - E.g., Squared Loss

Recap: Two Basic ML Problems

- **Classification** $f(x \mid w, b) = \text{sign}(w^T x - b)$
  - Predict which class an example belongs to
  - E.g., spam filtering example
- **Regression** $f(x \mid w, b) = w^T x - b$
  - Predict a real value or a probability
  - E.g., probability of being spam
- **Highly inter-related**
  - Train on Regression => Use for Classification

Recap: Supervised Learning Recipe

- Trainir $x \in R^D$ $y \in \{-1, +1\}$
- Mode **Linear Models**
- Loss F **Squared Loss**
- Learning Objective: $\arg\min_{w,b} \sum_{i=1}^{N} L(y_i, f(x_i|w,b))$

Congratulations!
You now know the basic steps to training a model!

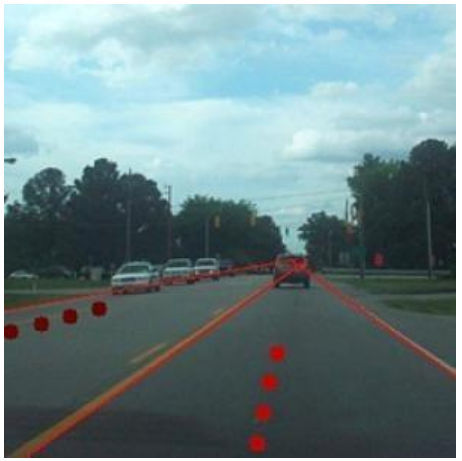
But is your model any good?

Optimization Problem

Example: Self-Driving Cars

Basic Setup

- Mounted cameras
- Use image features

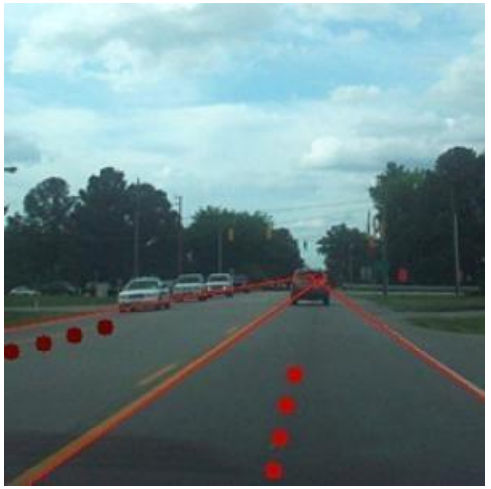


- Human demonstrations

- f(x|w) = steering angle
- Learn on training set

Overfitting

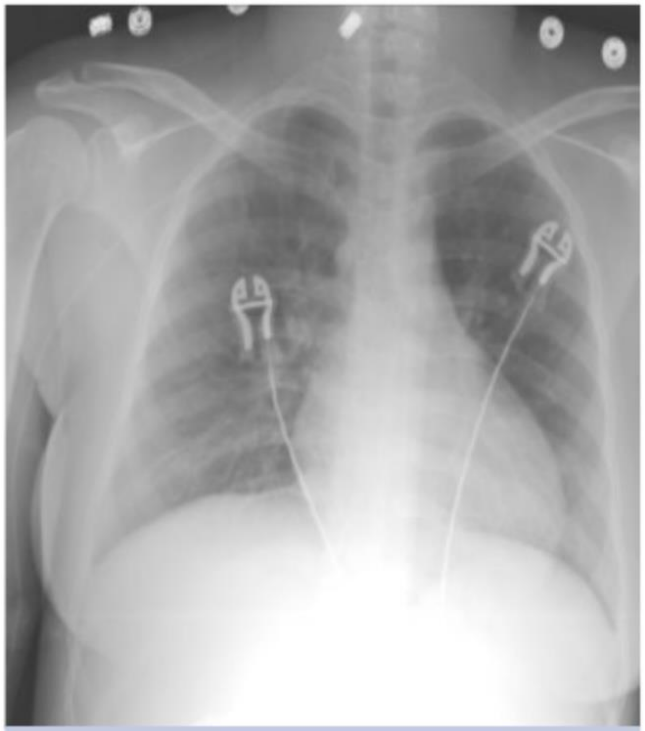
- Very accurate model



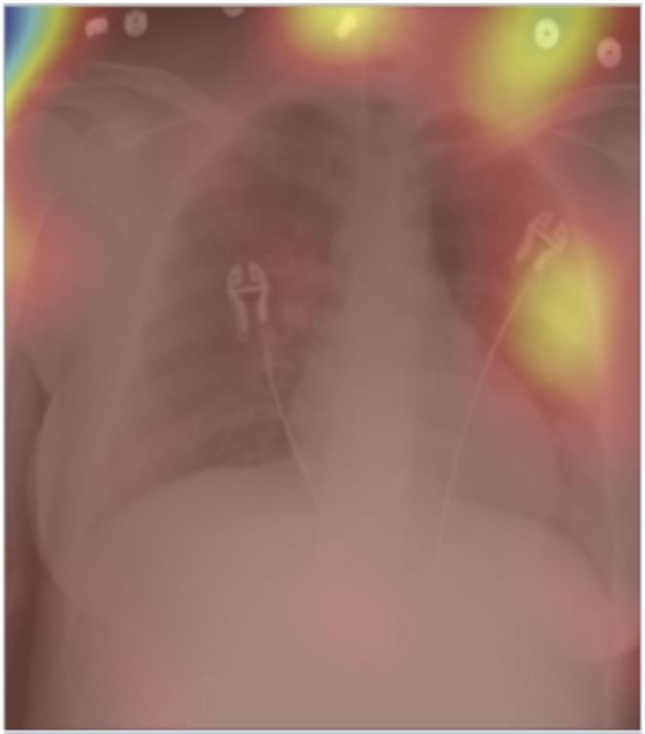
- But crashed on live test!
- Model w only cared about staying between two green patches

Overfitting Happens A Lot!

Example: Classifying Chest X-Ray vs Abdominal X-Ray



Classifier Confidence = 100%
Prediction: “chest X-ray”



Heat-Map
“where the AI is looking”

Src: https://www.acrdsi.org/DSIBlog/2022/04/11/Overfitting-and-Generalization-of-AI-in-Medical-Imaging

Overfitting in LLMs

A hunter leaves his tent. He travels 5 steps south, 5 steps east, 5 steps north, arriving back at his tent. He sees a brown bear inside. What color was the bear

The bear was **white** 🐻‍❄️

Reason:
For the hunter to walk 5 steps **south**, then 5 **east**, then 5 **north** and end up back at his tent, his tent must be at (or extremely close to) the **North Pole**. That's the only place on Earth where this path brings you back to your starting point.

The only bears that live at the North Pole region are **polar bears**, which are white—so the "brown bear" detail is there to mislead you.

(GPT 5.2, Jan 6th 2026)

https://chatgpt.com/share/695da833-9e98-800f-92d8-3c5336006691

Test Error

- **"True" distribution:** P(x,y)

  "All possible emails"

  – Unknown to us

- **Train:** f(x) = y

  – Using training data: $S = \{(x_i, y_i)\}_{i=1}^{N}$

  – Sampled identically and independently from P(x,y)

- **Test Error:**

  Prediction Loss on all possible emails

$$L_P(f) = E_{(x,y)\sim P(x,y)}\left[L(y, f(x))\right]$$

- **Overfitting:** Test Error >> Training Error

Test Error

- **Test Error:**

$$L_P(f) = E_{(x,y)\sim P(x,y)}\left[L(y, f(x))\right]$$

- **Treat f_S as random variable:** (randomness over *S*)

$$f_S = \arg\min_{w,b} \sum_{(x,y)\in S} L(y, f(x|w,b))$$

- **Expected Test Error:**

$$E_S[L_P(f_S)] = E_S\left[E_{(x,y)\sim P(x,y)}[L(y, f_S(x))]\right]$$

Bias-Variance Decomposition

$$E_S[L_P(f_S)] = E_S\left[E_{(x,y)\sim P(x,y)}[L(y, f_S(x))]\right]$$

- For squared error:

$$E_S[L_P(f_S)] = E_{(x,y)\sim P(x,y)}\left[E_S\left[\left(f_{S(x)} - F(x)\right)^2\right] + (F(x) - y)^2\right]$$

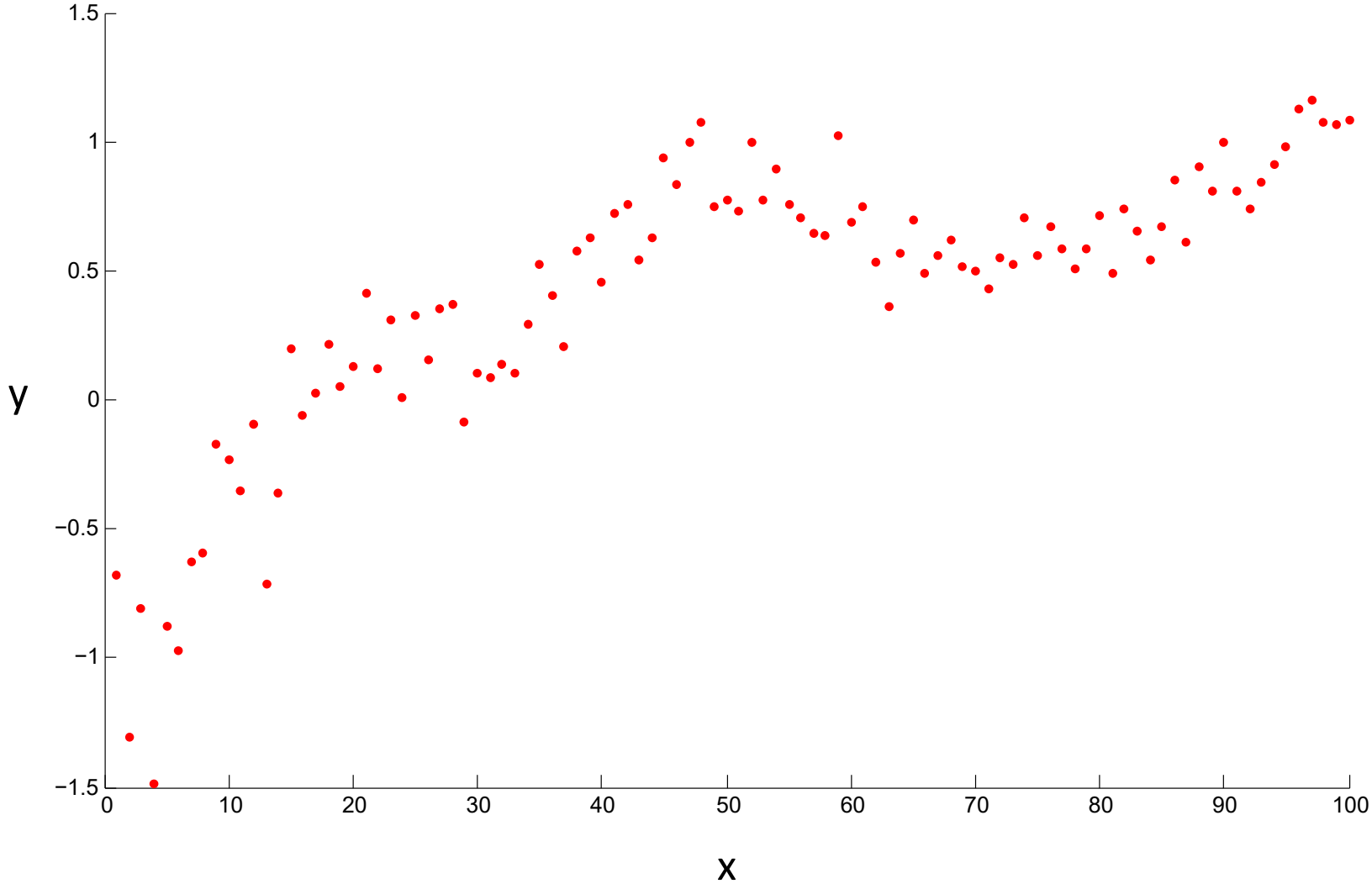
Variance Term

Bias Term

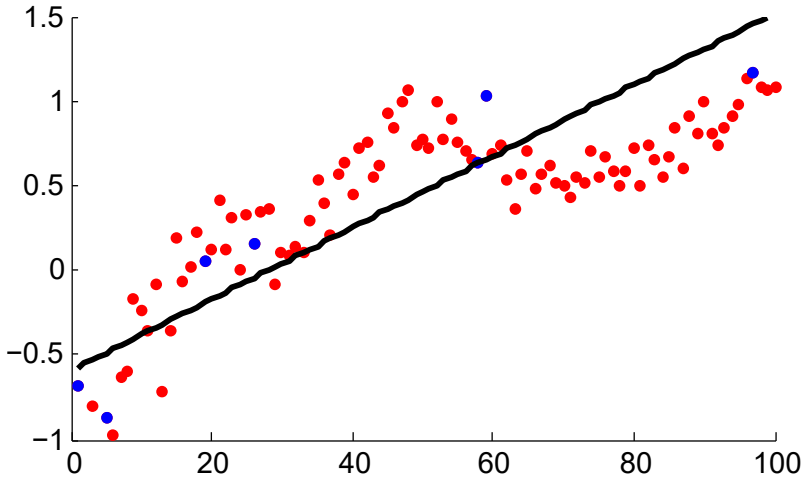
$$\mathsf{F}(x) = E_S[f_S(x)]$$

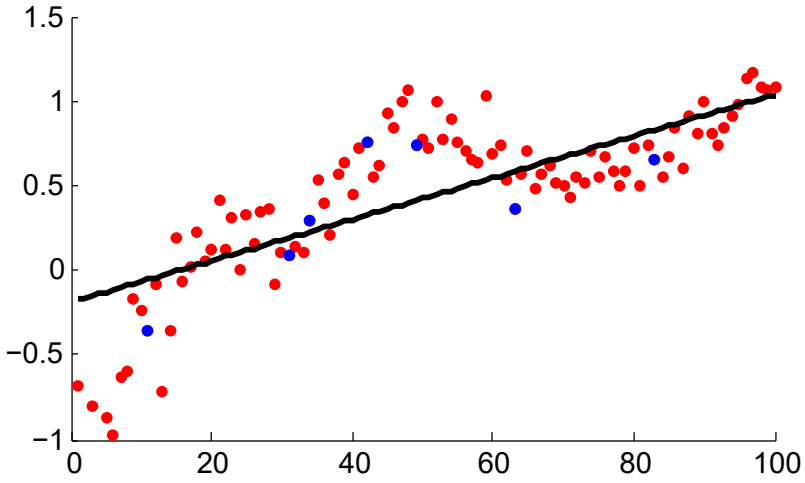
"Average prediction on x"

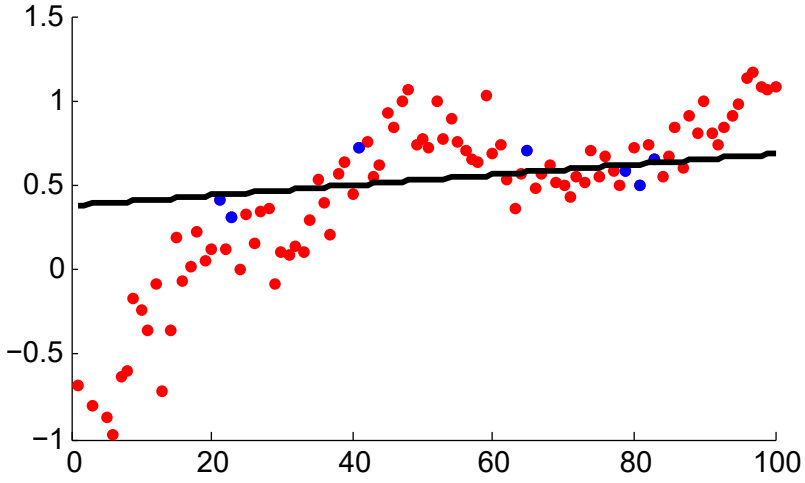
Example P(x,y)

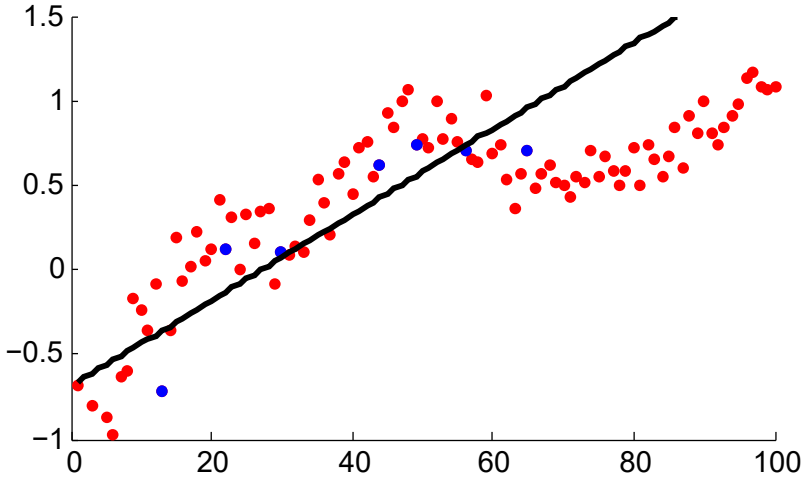


$f_S(x)$ Linear

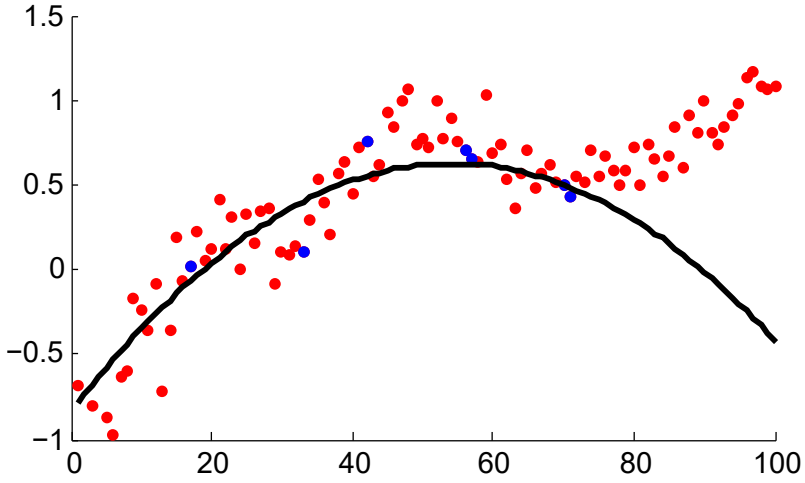

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

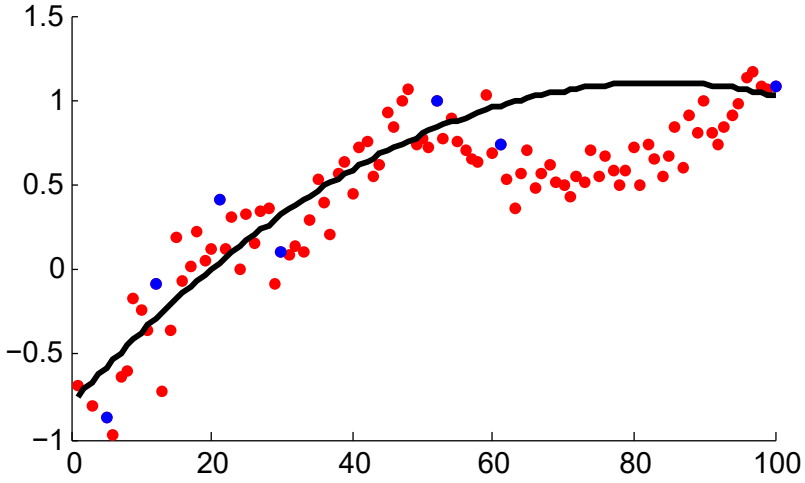

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

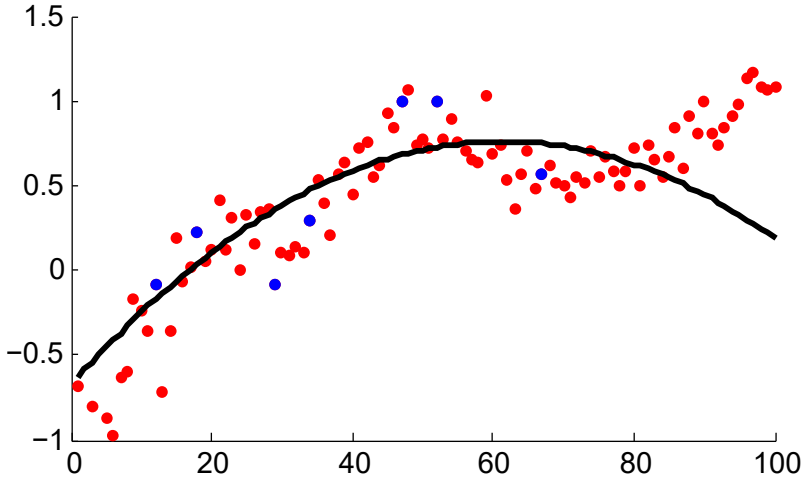

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

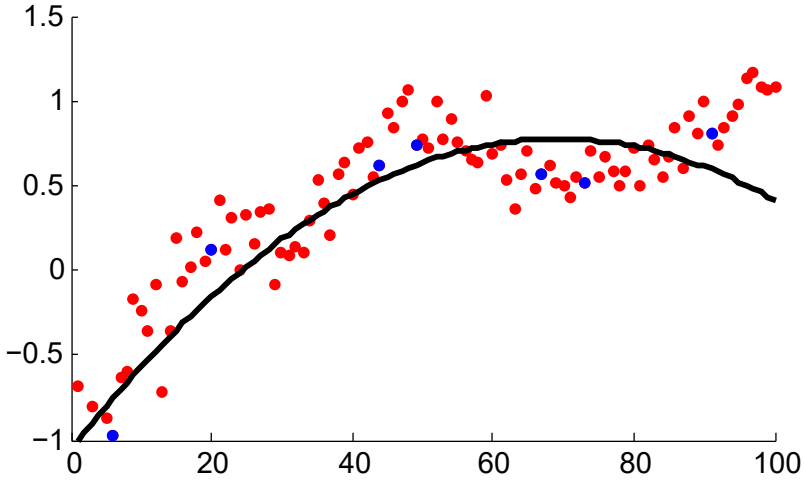

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

$f_S(x)$ Quadratic

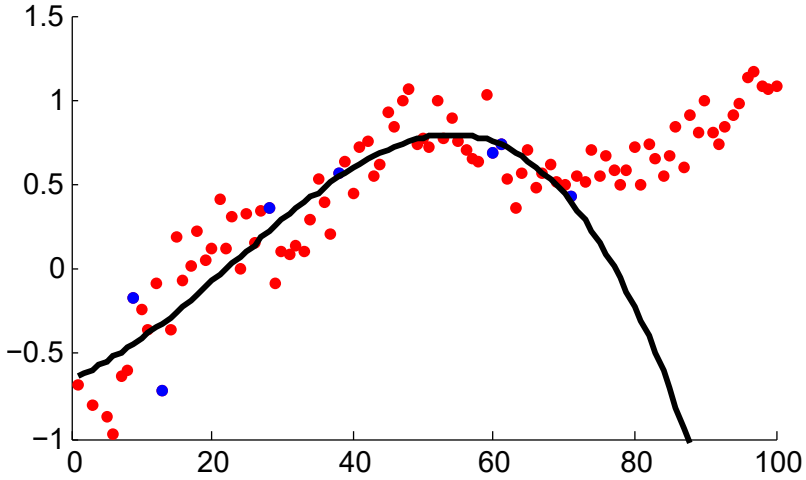


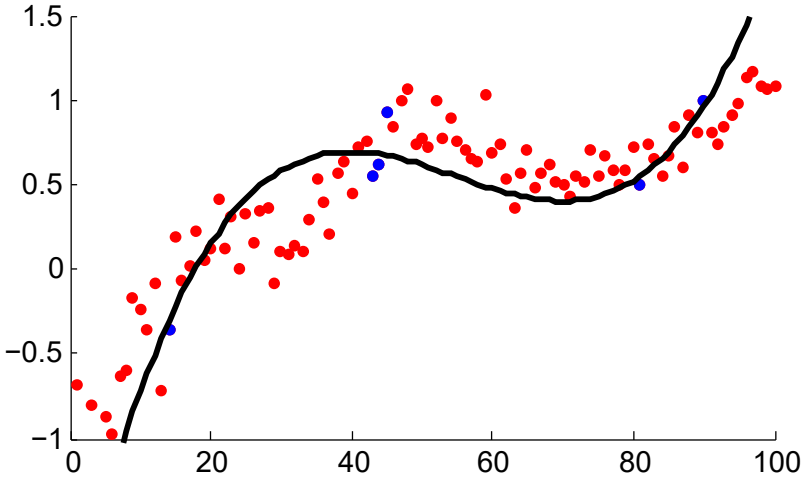


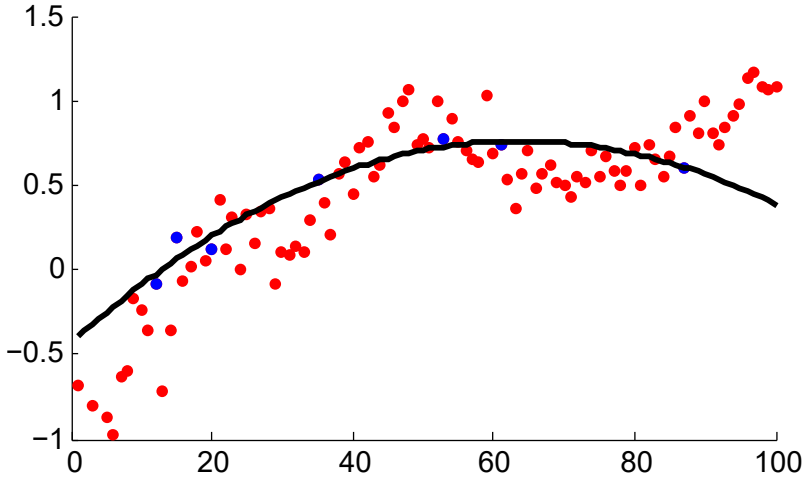


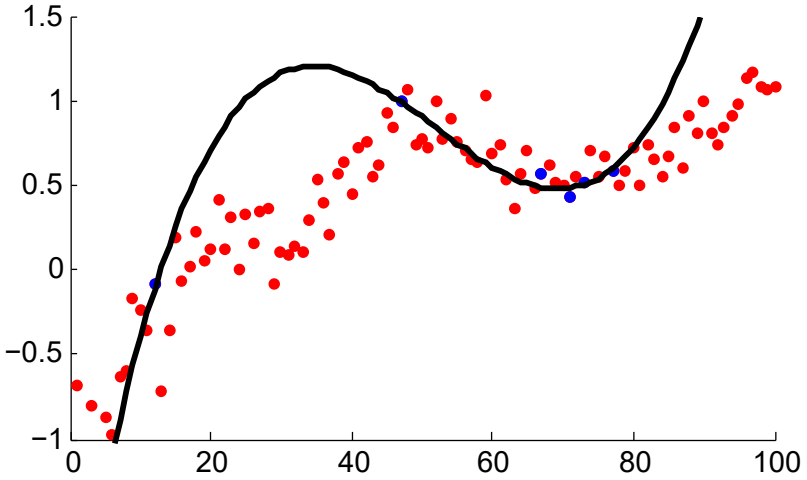


$f_S(x)$ Cubic

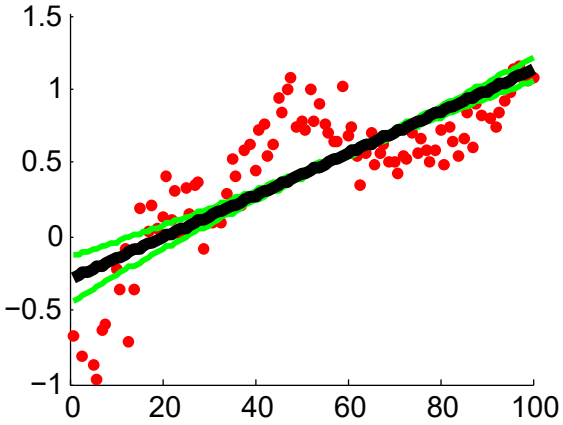

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

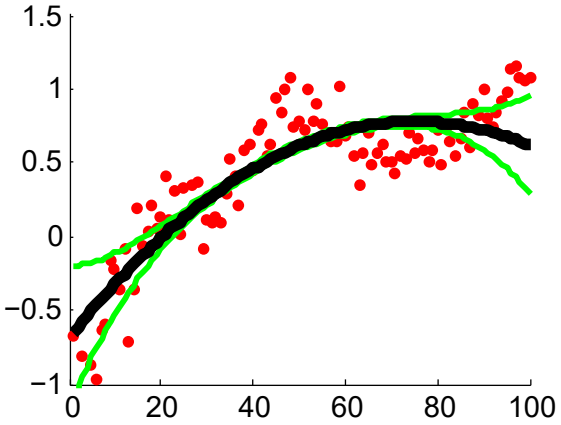

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

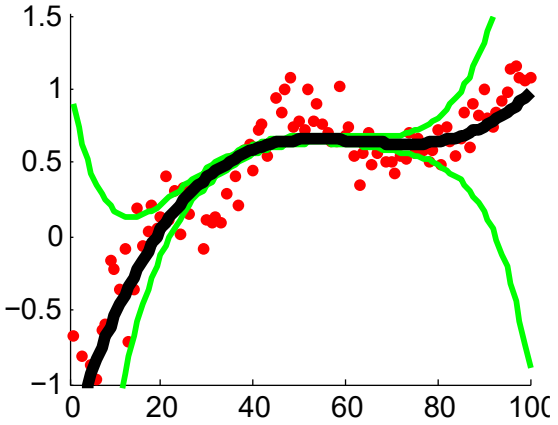

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

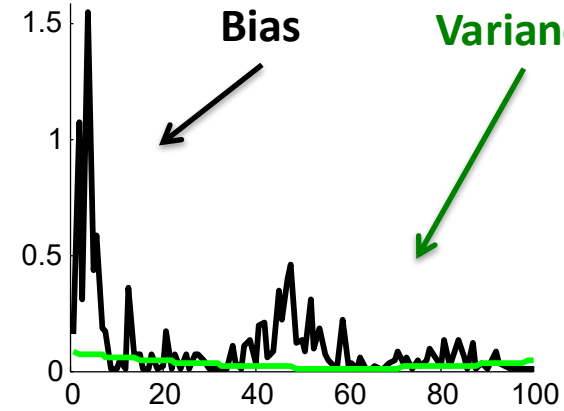

1.5
1
0.5
0
−0.5
−1
0
20
40
60
80
100

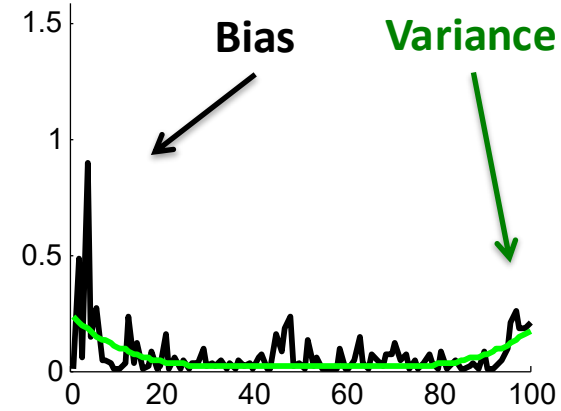
Bias-Variance Trade-off

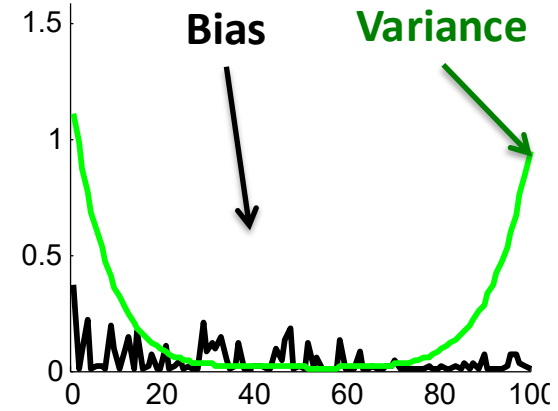




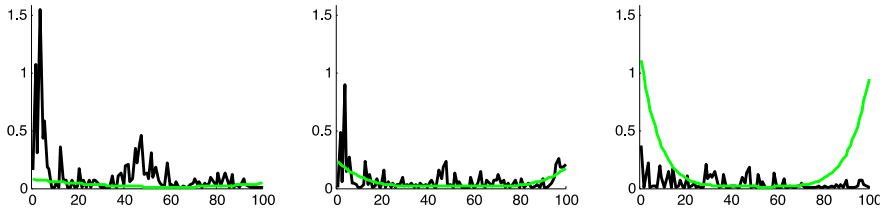






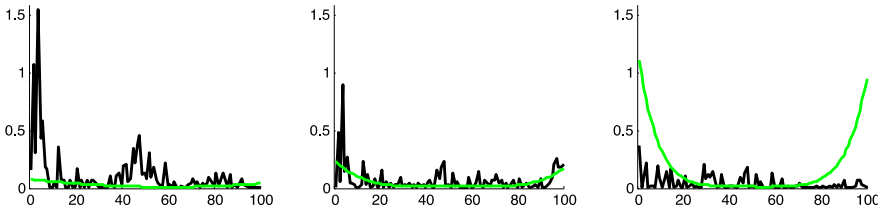


Overfitting vs Underfitting



- High variance implies **overfitting**
  - Model class unstable
  - Variance increases with model complexity
  - Variance reduces with more training data.
- High bias implies **underfitting**
  - Even with no variance, model class has high error
  - Bias decreases with model complexity
  - Independent of training data size

Model Selection



- Finite training data
-
-
-

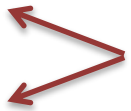
But we can't measure test error directly!

(Don't have the whole distribution.)

error

Use a Validation Set!

- Split data to Training Set and Validation Set
- Train model on Training Set
- Evaluate on Validation Set

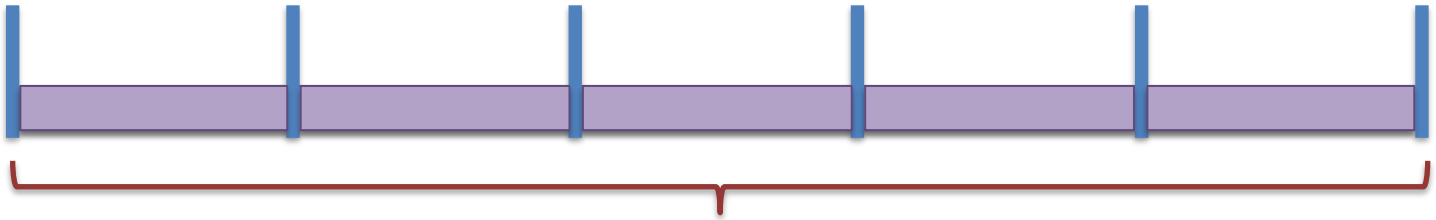


Keep training and evaluation separate!

- What's wrong with this?
  - **If dataset small, validation set small!**

5-Fold Cross Validation

(one validation option)



- Split data into 5 equal partitions

Training & Validation set sampled **independently** from same distribution

- Allows using all data as validation

Complete Pipeline
(Supervised Learning)

$$S = \{(x_i, y_i)\}_{i=1}^{N}$$

Training Data

$$f(x \mid w, b) = w^T x - b$$

Model Class(es)

$$L(a, b) = (a - b)^2$$

Loss Function

$$\underset{w,b}{\operatorname{argmin}} \sum_{i=1}^{N} L\left(y_i, f(x_i \mid w, b)\right)$$

Cross Validation & Model Selection

Profit!

Desiderata for Modern AI

- In deployed settings, test set is constantly evolving
  - With time
  - With new users
  - With new use cases

- Need to redo this pipeline repeatedly

Next Lecture

- Perceptron
- Stochastic Gradient Descent

- Recitation on Thursday
  - Introduction to Python